

01 UART Specification

- [SMC UART 16550 — Detailed Test Plan](#)
 - [Contents](#)
 - [1. Objectives](#)
 - [2. Scope \(in / out\)](#)
 - [2.1 In scope](#)
 - [2.2 Out of scope](#)
 - [3. Verification strategy](#)
 - [4. Environment](#)
 - [5. Test bench architecture](#)
 - [5.1 Transaction helpers](#)
 - [6. Stimulus and checking primitives](#)
 - [7. Feature to test traceability matrix](#)
 - [8. Detailed test cases](#)
 - [TC-1 — Register reset values and field semantics](#)
 - [TC-2 — DLAB muxing](#)
 - [TC-3 — Basic transmit](#)
 - [TC-4 — Basic receive](#)
 - [TC-5 — FIFO overflow](#)
 - [TC-6 — Non-FIFO mode](#)
 - [TC-7 — RX FIFO trigger level](#)
 - [TC-8 — Interrupt priority and IIR](#)
 - [TC-9 — Line-status errors](#)
 - [TC-10 — Modem status and delta bits](#)
 - [TC-11 — Loopback](#)
 - [TC-12 — DMA handshake](#)
 - [TC-13 — Interrupt test register](#)
 - [TC-14 — RX timeout](#)
 - [TC-15 — Reset during operation](#)
 - [TC-16 — Bus error responses \(negative\)](#)
 - [TC-17 — Debug transport](#)
 - [9. Coverage plan](#)
 - [10. Negative-test catalogue](#)
 - [11. Regression workflow](#)
 - [12. Pass / fail criteria and exit conditions](#)
 - [13. Future tests and open work](#)

SMC UART 16550 — Detailed Test Plan

Document: 03_UART_Test_Plan.md **Module under test (MUT):** smc::uart (peripherals/uart/) **Reference test bench:** peripherals/uart/test/uart_tb.cpp **RTL reference:** hw/comp/uart_16550/rtl/ and hw/comp/uart_16550/data/registers/rdl/ **Status:** Design phase — defines the planned verification strategy and test list **Companion docs:** - 01_UART_Specification.md — externally-observable behaviour - 02_UART_LowLevel_Design.md — implementation contract

Contents

1. [Objectives](#)
 2. [Scope \(in / out\)](#)
 3. [Verification strategy](#)
 4. [Environment](#)
 5. [Test bench architecture](#)
 6. [Stimulus and checking primitives](#)
 7. [Feature to test traceability matrix](#)
 8. [Detailed test cases](#)
 9. [Coverage plan](#)
 10. [Negative-test catalogue](#)
 11. [Regression workflow](#)
 12. [Pass / fail criteria and exit conditions](#)
 13. [Future tests and open work](#)
-

1. Objectives

The UART test plan must demonstrate that the SystemC functional model:

1. **Conforms** to the 16550 programming model defined in 01_UART_Specification.md and the ground-truth uart_16550_*.rdl register maps (register layout, field semantics, DLAB muxing).
 2. **Implements correctly** every flow described in the specification: transmit, receive, FIFO management, interrupt priority/identification, modem control, loopback, DMA handshaking, and reset.
 3. **Honours** the TLM-2.0 contract in 01_UART_Specification.md §10 (alignment, width, error responses, debug back door, no DMI).
 4. **Is robust** against negative stimulus (misaligned, oversize, out-of-window, bad command, FIFO overflow/underflow).
 5. **Behaves correctly under loosely-timed temporal decoupling** with a quantum keeper.
-

2. Scope (in / out)

2.1 In scope

- Self-checking SystemC unit tests for every register and observable behaviour.
- Stimulus through b_transport (the production access path) driven by a quantum keeper.

- RX stimulus through the `inject_rx_char()` back door.
- Back-door inspection through `dbg_*` and `transport_dbg`.
- Branch and line coverage of `src/uart.cpp` (target $\geq 95\%$).

2.2 Out of scope

- Bit-level serial timing / baud-rate accuracy (not modelled — see spec §13).
- Glitch/noise filtering, clock-domain crossing behaviour.
- Multi-UART serial interconnect.
- Firmware co-simulation (covered at the platform level, not here).

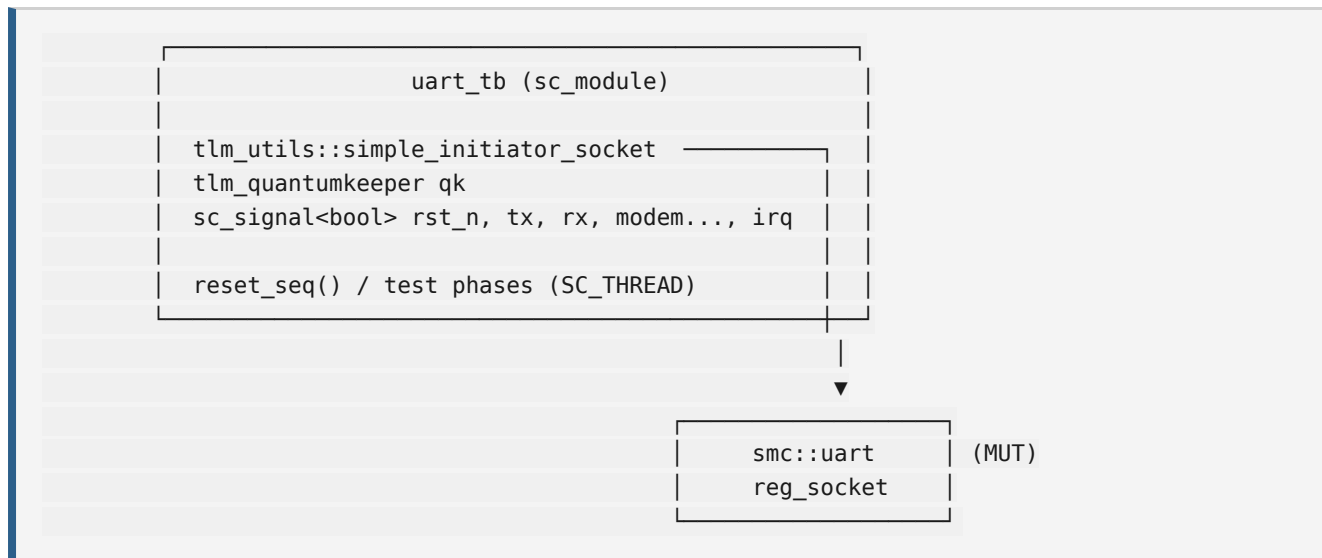
3. Verification strategy

Layer	Technique
Register access	Directed read/write of every register; check reset values, RW/RO/WO semantics, reserved bits.
Data flow	Transmit characters, pop via <code>dbg_tx_pop()</code> ; inject characters, read via RBR.
FIFO	Fill/drain to depth boundaries; overflow and underflow; FIFO vs non-FIFO mode.
Interrupts	Trigger each source individually, verify IIR priority encode and <code>irq_o</code> .
Modem/loopback	Drive modem inputs, check MSR + delta bits; exercise system and line loopback.
DMA	Mode 0 and Mode 1 RX/TX FSM transitions on <code>rxrdy_o</code> / <code>txrdy_o</code> .
Bus contract	Misaligned/oversize/out-of-window/bad-command transactions → correct TLM response.
Timing	Run all of the above through a quantum keeper with a non-trivial global quantum.
Coverage	<code>gcovr</code> (GCC) / <code>llvm-cov</code> (Clang) over <code>src/</code> only.

4. Environment

Item	Value
Language standard	C++20 (per repo build standard)
SystemC	3.0.2 Accellera (cxx20 build)
CCI	OSCI CCI (cxx20 build)
Compilers	GCC ≥ 11 / Apple Clang 17
Coverage tools	<code>gcovr</code> (GCC), <code>llvm-cov</code> + <code>llvm-profdata</code> (Clang)
Build	out-of-source CMake (<code>build/</code> , <code>cov_build/</code>)
Runner	<code>peripherals/uart/run_tests.sh</code>

5. Test bench architecture



The test bench is a single `SC_THREAD` that walks through numbered phases, each a group of related checks. A shared `CHECK(cond, msg)` macro increments a pass/fail counter and prints a diagnostic on failure. The thread ends with `sc_stop()` and a summary line.

5.1 Transaction helpers

```

uint32_t rd(uint64_t off);           // 32-bit read via b_transport + qk
void wr(uint64_t off, uint32_t v); // 32-bit write via b_transport + qk
void rd_expect_err(uint64_t off, tlm::tlm_response_status exp);
  
```

Each helper builds a `tlm_generic_payload`, passes `qk.get_local_time()` as the delay, calls `b_transport`, then `qk.set_and_sync(delay)`.

6. Stimulus and checking primitives

Primitive	Description
<code>wr(off, val) / rd(off)</code>	Production-path register access through the QK.
<code>inject_rx_char(ch, p, f, b)</code>	Deliver an RX character with optional error flags.
<code>dbg_tx_pop(ch)</code>	Pop the next transmitted character; verify TX data.
<code>dbg_tx_count() / dbg_rx_count()</code>	Verify FIFO occupancy.
<code>dbg_reg(off)</code>	Side-effect-free register peek for assertions.
modem signal drive	Set <code>cts_ni/dsr_ni/ri_ni/dcd_ni</code> and <code>wait(SC_ZERO_TIME)</code> for the method to run.
<code>CHECK(cond, msg)</code>	Self-checking assertion with pass/fail accounting.

7. Feature to test traceability matrix

Spec feature	Spec ref	Test case(s)
Register reset values	§7, §9	TC-1
RW/RO/WO field semantics	§7	TC-1, TC-2
DLAB muxing (DLL/DLM vs THR/RBR/IER)	§7.1/7.2	TC-2
Transmit + THRE/TEMT	§8.1	TC-3
Receive + DR	§8.2	TC-4
FIFO fill/drain/overflow	§8.2, §8.3	TC-5
FIFO vs non-FIFO mode	§3, §8.3	TC-6
RX FIFO trigger level (FCR+ECR)	§7.3	TC-7
Interrupt priority + IIR encode	§6.6	TC-8
IIR read clears data-ready	§7.3	TC-8
Line-status errors (OE/PE/FE/BI)	§11.1	TC-9
Modem status + delta bits	§7.3	TC-10
System loopback	§8.4	TC-11
Line loopback	§8.4	TC-11
DMA Mode 0	§6.7	TC-12
DMA Mode 1 FSMs	§6.7	TC-12
ITR force-interrupt	§7.3	TC-13
RX timeout interrupt	§6.5	TC-14
Reset behaviour	§9	TC-15
Bus error responses	§10.2	TC-16 (negative)
<code>transport_dbg</code> no side effects	§10.3	TC-17
Quantum-keeper temporal decoupling	LLD §10	All (driven through QK)

8. Detailed test cases

TC-1 — Register reset values and field semantics

- **Setup:** Apply reset, de-assert.
- **Steps:** Read every register; verify `LSR = 0x60`, `IIR[0]=1`, all control regs = 0. Write/read-back SCR (full RW), attempt to write RO bits and confirm they don't stick.
- **Pass:** All reset values and access types match §7.

TC-2 — DLAB muxing

- **Steps:** With DLAB=0, write IER at 0x04 and read back. Set LCR.DLAB=1; write DLL/DLM at 0x00/0x04; read back. Clear DLAB; confirm 0x04 again reads IER (not DLM).
- **Pass:** 0x00/0x04 decode correctly in both DLAB states.

TC-3 — Basic transmit

- **Steps:** Configure 8N1, enable FIFO. Write 4 bytes to THR. Pop via `dbg_tx_pop()`. Check LSR.THRE/TEMT rise after drain.
- **Pass:** Transmitted bytes match written bytes in order; THRE=TEMT=1 when empty.

TC-4 — Basic receive

- **Steps:** `inject_rx_char()` 4 bytes. Verify LSR.DR=1. Read RBR 4 times; verify data and that DR clears when empty.
- **Pass:** Received bytes match injected bytes; DR tracks occupancy.

TC-5 — FIFO overflow

- **Steps:** Inject `rx_fifo_depth + 2` characters. Verify LSR.OE set and only `depth` characters are retrievable.
- **Pass:** Overrun flagged; excess characters dropped.

TC-6 — Non-FIFO mode

- **Steps:** Leave FCR.FIFO_ENABLE=0. Inject 2 chars; verify the second overwrites/overruns the single RBR byte. Write 2 THR bytes; verify single-byte holding behaviour.
- **Pass:** Single-byte RBR/THR semantics hold; FIFOS_ENABLED reads 0.

TC-7 — RX FIFO trigger level

- **Steps:** Program trigger level via FCR.RCVR_TRIGGER + ECR.RCVR_TRIGGER_MS2B for several values (1, 4, 8, 32). Inject up to the threshold; verify data-ready interrupt asserts at the configured level.
- **Pass:** Interrupt asserts at exactly the programmed trigger level.

TC-8 — Interrupt priority and IIR

- **Steps:** Enable all interrupts (IER=0x1F). Create simultaneous conditions (e.g. line status + data ready); read IIR and verify the higher-priority ID is reported. Read IIR / drain to clear and verify the next source surfaces.
- **Pass:** IIR.INTERRUPT_ID matches the §6.6 priority order; INTERRUPT_PENDING active-low.

TC-9 — Line-status errors

- **Steps:** Inject characters with parity, framing, and break flags. Read LSR; verify PE/FE/BI set and ERROR_IN_RCVR_FIFO set; verify they clear on read and the RLS interrupt fires when IER.ELSI=1.
- **Pass:** Each error bit behaves per §11.1; `err_o` reflects the aggregate.

TC-10 — Modem status and delta bits

- **Steps:** Drive `cts_ni/dsr_ni/ri_ni/dcd_ni`; read MSR and verify CTS/DSR/RI/DCD levels (active-high inversion). Toggle inputs; verify DCTS/DDSR/DDCD set and TERI on RI 1→0; verify delta bits clear on MSR read; verify modem-status interrupt with IER.EDSSI.
- **Pass:** MSR levels and deltas match §7.3.

TC-11 — Loopback

- **Steps:** Set MCR.LOOP; transmit a byte; verify it appears on the RX path and modem status reflects MCR bits. Set MCR.LINE_LOOPBACK; verify rx_i routes to tx_o and MSR inputs are deasserted.
- **Pass:** Both loopback modes behave per §8.4; line loopback takes precedence.

TC-12 — DMA handshake

- **Steps:** Mode 0: verify rxrdy_o tracks RX non-empty and txrdy_o tracks TX not-full. Mode 1 (FIFO on): inject to watermark and verify RXRDY asserts until drained; fill TX and verify TXRDY deasserts until empty.
- **Pass:** FSM transitions match §6.7.

TC-13 — Interrupt test register

- **Steps:** For each ITR bit, set it (with matching IER enable) and verify the corresponding IIR ID and irq_o; clear and verify release.
- **Pass:** Every forced interrupt source is observable and clears.

TC-14 — RX timeout

- **Steps:** Enable FIFO + data-ready interrupt. Inject a character below trigger level; advance time past the timeout. Verify IIR reports 0x6 and draining clears it.
- **Pass:** Timeout interrupt fires and clears per §6.5.

TC-15 — Reset during operation

- **Steps:** Load TX/RX FIFOs, set control regs, raise interrupts. Assert reset. Verify all registers/FIFOs return to reset state and all outputs deassert.
- **Pass:** Post-reset state matches §9.

TC-16 — Bus error responses (negative)

- See negative-test catalogue §10.

TC-17 — Debug transport

- **Steps:** Inject characters; use transport_dbg/dbg_reg to read RBR, IIR, LSR, MSR; verify the RX FIFO is **not** popped and sticky/delta bits are **not** cleared.
 - **Pass:** Debug reads have zero side effects.
-

9. Coverage plan

Metric	Target	Tool
Line coverage of <code>src/uart.cpp</code>	≥ 95%	<code>gcovr / llvm-cov</code>
Branch coverage of <code>src/uart.cpp</code>	≥ 95%	<code>gcovr --exclude-throw-branches --exclude-unreachable-branches / llvm-cov</code>
Function coverage	100% of public + internal methods	both

Coverage is measured over `src/` only (the test bench and headers are excluded). Genuinely unreachable defensive paths are marked `LCOV_EXCL_LINE` with a justifying comment. The runner prints a per-file summary and lists any uncovered lines in `uart.cpp`.

10. Negative-test catalogue

ID	Stimulus	Expected response
N1	Read/write at <code>addr >= 0x100</code> (out of window)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
N2	Unaligned access (<code>addr & 0x3 != 0</code>)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
N3	<code>data_length != 4</code> (e.g. 1 or 8)	<code>TLM_BURST_ERROR_RESPONSE</code>
N4	Command other than READ/WRITE	<code>TLM_COMMAND_ERROR_RESPONSE</code>
N5	Decode miss inside window (e.g. <code>0x28..0xFC</code>)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
N6	Read RBR when RX FIFO empty	Returns <code>0x00</code> , no underflow, DR stays 0
N7	Write THR when TX FIFO full	Byte silently dropped, no crash
N8	Inject RX char when FIFO full	LSR.OE set, char dropped
N9	<code>get_direct_mem_ptr</code> request	Returns false (DMI never granted)
N10	Set baud divisor = 0, then transmit	TX/RX disabled; no transmit occurs

11. Regression workflow

```
# From peripherals/uart/
SYSTEMC_HOME=/path/to/systemc-cxx20 \
CCI_HOME=/path/to/cci-cxx20 \
./run_tests.sh                # build + run unit tests

./run_tests.sh --coverage      # build instrumented, run, emit coverage report
```

`run_tests.sh` selects `gcovr` for GCC builds and `llvm-cov/llvm-profdata` for Clang builds, reporting line + branch coverage for `src/uart.cpp` and listing any uncovered lines. The UART is also wired into

`smc/run_all_smc_tests.sh` for the full regression.

12. Pass / fail criteria and exit conditions

The test run **passes** iff:

1. Every `CHECK` passes (failure counter = 0).
2. The test bench reaches its final summary and calls `sc_stop()` cleanly (no SystemC errors, no unbound-port or multiple-driver diagnostics).
3. No transaction returns an unexpected TLM response.
4. Line and branch coverage of `src/uart.cpp` are both $\geq 95\%$.

Any `CHECK` failure, SystemC error/fatal, or coverage shortfall fails the run.

13. Future tests and open work

- **Approximately-timed profile:** if an AT variant is added (per LLD §15), extend TC-3/ TC-4 to assert per-character serial delays.
- **Randomised soak:** constrained-random mix of reads/writes/injects with a scoreboard model of the FIFOs.
- **Firmware-level test:** run an unmodified console driver against the model through the full fabric path at the platform level.
- **Cross-UART loopback:** once a serial-line model exists, wire two UART instances and verify end-to-end character transfer.